

IPA ASSIGNMENT - 16

Name - Nishant Nahar

Roll No - 241551078

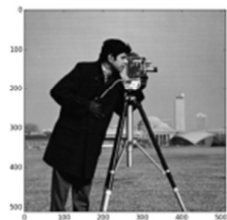
1. All types of Noises - Gaussian, Salt and Pepper, Gamma, Poisson, Uniform, Rayleigh Noise

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
img = cv2.imread("img_3.jpeg", cv2.IMREAD_GRAYSCALE)
if img is None: raise FileNotFoundError("Image not found. Check the file path.")
img = img.astype(np.float32) / 255.0
def gaussian_noise(image, mean=0, sigma=0.1):
    noise = np.random.normal(mean, sigma, image.shape)
    noisy = image + noise
    return np.clip(noisy, 0, 1)
def salt_pepper_noise(image, amount=0.05):
    noisy = image.copy()
    salt = np.random.random(image.shape) < amount / 2
    noisy[salt] = 1
    pepper = np.random.random(image.shape) < amount / 2
    noisy[pepper] = 0
    return noisy
def speckle_noise(image, sigma=0.2):
    noise = np.random.randn(*image.shape) * sigma
    noisy = image + image * noise
    return np.clip(noisy, 0, 1)
def poisson_noise(image):
    vals = 2**8
    noisy = np.random.poisson(image * vals) / vals
    return np.clip(noisy, 0, 1)
def uniform_noise(image, low=-0.1, high=0.1):
    noise = np.random.uniform(low, high, image.shape)
    noisy = image + noise
    return np.clip(noisy, 0, 1)
def rayleigh_noise(image, scale=0.1):
    noise = np.random.rayleigh(scale, image.shape)
    noise = noise - np.mean(noise)
    noisy = image + noise
    return np.clip(noisy, 0, 1)
gaussian = gaussian_noise(img)
salt_pepper = salt_pepper_noise(img)
speckle = speckle_noise(img)
poisson = poisson_noise(img)
uniform = uniform_noise(img)
rayleigh = rayleigh_noise(img)
images = [img, gaussian, salt_pepper, speckle, poisson, uniform, rayleigh]
titles = [
    "Original Image", "Gaussian Noise", "Salt & Pepper Noise", "Speckle Noise"
    "Poisson Noise", "Uniform Noise", "Rayleigh Noise",]
plt.figure(figsize=(12, 8))
for i in range(7):
    plt.subplot(2, 4, i + 1)
    plt.imshow(images[i], cmap="gray")
```

```
plt.title(titles[i])
plt.axis("off")
plt.tight_layout()
plt.show()
```

OUTPUT

Original Image



Gaussian Noise



Salt & Pepper Noise



Speckle Noise



Poisson Noise



Uniform Noise



Rayleigh Noise



2. SNR Before and After

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
img = cv2.imread("img_1.jpeg", cv2.IMREAD_GRAYSCALE)
if img is None: raise FileNotFoundError("Image not found. Check the image path.")
img = img.astype(np.float32) / 255.0
def gaussian_noise(image, mean=0, sigma=0.1):
    noise = np.random.normal(mean, sigma, image.shape)
    noisy = image + noise
    return np.clip(noisy, 0, 1)
def salt_pepper_noise(image, amount=0.05):
    noisy = image.copy()
    salt = np.random.random(image.shape) < amount / 2
    pepper = np.random.random(image.shape) < amount / 2
    noisy[salt] = 1
    noisy[pepper] = 0
    return noisy
def speckle_noise(image, sigma=0.2):
    noise = np.random.randn(*image.shape) * sigma
    noisy = image + image * noise
    return np.clip(noisy, 0, 1)
def poisson_noise(image):
    vals = 256
```

```

    noisy = np.random.poisson(image * vals) / vals
    return np.clip(noisy, 0, 1)
def uniform_noise(image, low=-0.1, high=0.1):
    noise = np.random.uniform(low, high, image.shape)
    noisy = image + noise
    return np.clip(noisy, 0, 1)
def rayleigh_noise(image, scale=0.1):
    noise = np.random.rayleigh(scale, image.shape)
    noise = noise - np.mean(noise)
    noisy = image + noise
    return np.clip(noisy, 0, 1)
gaussian = gaussian_noise(img)
salt_pepper = salt_pepper_noise(img)
speckle = speckle_noise(img)
poisson = poisson_noise(img)
uniform = uniform_noise(img)
rayleigh = rayleigh_noise(img)
def calculate_snr(original, noisy):
    signal_power = np.mean(original**2)
    noise = noisy - original
    noise_power = np.mean(noise**2)
    if noise_power == 0:
        snr = 10 * np.log10(signal_power / noise_power)
    return snr
snr_gaussian = calculate_snr(img, gaussian)
snr_sp = calculate_snr(img, salt_pepper)
snr_speckle = calculate_snr(img, speckle)
snr_poisson = calculate_snr(img, poisson)
snr_uniform = calculate_snr(img, uniform)
snr_rayleigh = calculate_snr(img, rayleigh)
print("Original Image SNR : Infinity dB")
print("Gaussian Noise SNR : {:.2f} dB".format(snr_gaussian))
print("Salt & Pepper SNR : {:.2f} dB".format(snr_sp))
print("Speckle Noise SNR : {:.2f} dB".format(snr_speckle))
print("Poisson Noise SNR : {:.2f} dB".format(snr_poisson))
print("Uniform Noise SNR : {:.2f} dB".format(snr_uniform))
print("Rayleigh Noise SNR : {:.2f} dB".format(snr_rayleigh))

```

```

-----
                        SNR RESULTS
-----
Original Image SNR : Infinity dB
Gaussian Noise SNR : 14.35 dB
Salt & Pepper SNR : 10.77 dB
Speckle Noise SNR : 14.23 dB
Poisson Noise SNR : 22.02 dB
Uniform Noise SNR : 19.04 dB
Rayleigh Noise SNR : 17.79 dB

```

3. All Mean - Geometric Harmonic and Contra harmonic Means

```

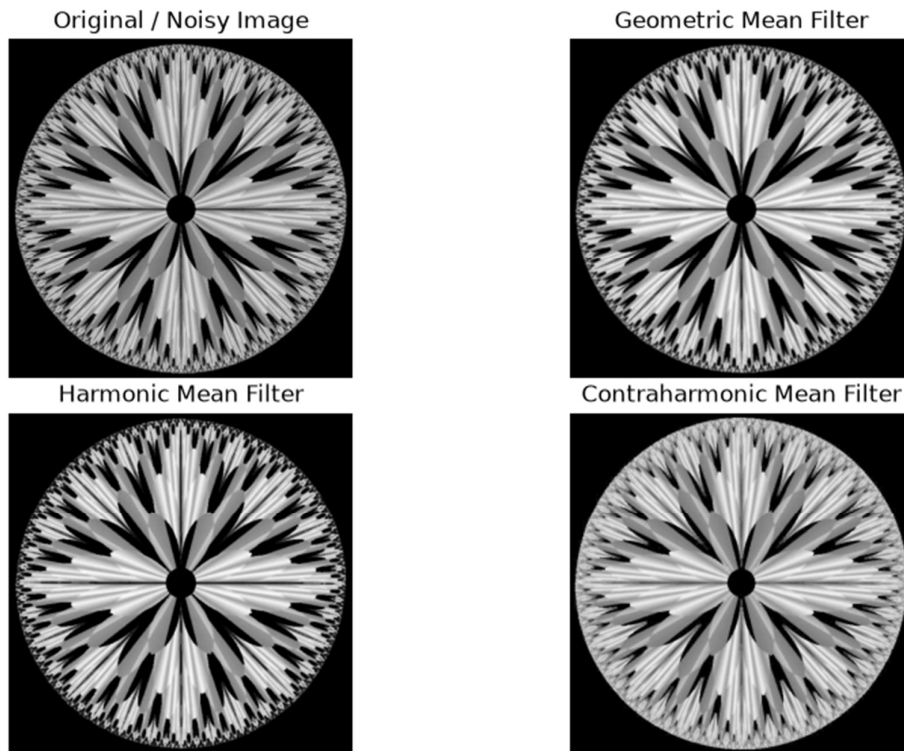
import cv2
import numpy as np
import matplotlib.pyplot as plt

```

```

img = cv2.imread("img_1.jpeg", cv2.IMREAD_GRAYSCALE)
if img is None:raise FileNotFoundError("Image not found")
def geometric_mean_filter(image, size=3):
    image = image.astype(np.float32) + 1e-5
    log_image = np.log(image)
    kernel = np.ones((size, size), np.float32) / (size * size)
    filtered = np.exp(cv2.filter2D(log_image, -1, kernel))
    return np.clip(filtered, 0, 255).astype(np.uint8)
def harmonic_mean_filter(image, size=3):
    image = image.astype(np.float32) + 1e-5
    kernel = np.ones((size, size), np.float32)
    numerator = size * size
    denominator = cv2.filter2D(1.0 / image, -1, kernel)
    filtered = numerator / denominator
    return np.clip(filtered, 0, 255).astype(np.uint8)
def contraharmonic_mean_filter(image, size=3, Q=1.5):
    image = image.astype(np.float32)
    kernel = np.ones((size, size), np.float32)
    numerator = cv2.filter2D(np.power(image, Q + 1), -1, kernel)
    denominator = cv2.filter2D(np.power(image, Q), -1, kernel)
    denominator = denominator + 1e-5
    filtered = numerator / denominator
    return np.clip(filtered, 0, 255).astype(np.uint8)
geometric = geometric_mean_filter(img, 3)
harmonic = harmonic_mean_filter(img, 3)
contraharmonic = contraharmonic_mean_filter(img, size=3, Q=1.5)
plt.figure(figsize=(12, 8))
plt.subplot(2, 2, 1)
plt.imshow(img, cmap="gray")
plt.title("Original / Noisy Image")
plt.axis("off")
plt.subplot(2, 2, 2)
plt.imshow(geometric, cmap="gray")
plt.title("Geometric Mean Filter")
plt.axis("off")
plt.subplot(2, 2, 3)
plt.imshow(harmonic, cmap="gray")
plt.title("Harmonic Mean Filter")
plt.axis("off")
plt.subplot(2, 2, 4)
plt.imshow(contraharmonic, cmap="gray")
plt.title("Contraharmonic Mean Filter")
plt.axis("off")
plt.tight_layout()
plt.show()

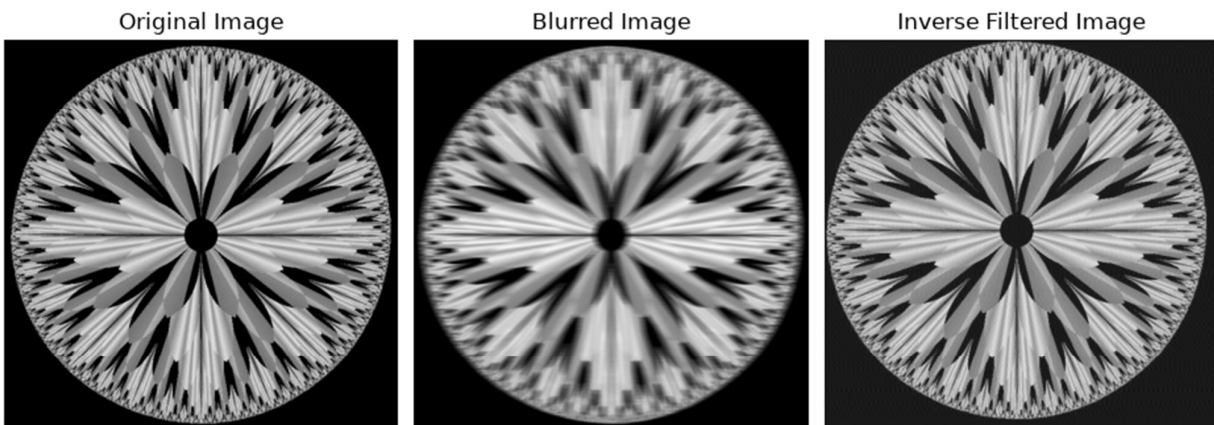
```



4. Inverse Filtering

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
img = cv2.imread("img_1.jpeg", cv2.IMREAD_GRAYSCALE)
if img is None: raise FileNotFoundError("Image not found")
img = img.astype(np.float32) / 255.0
def motion_blur_kernel(size=15):
    kernel = np.zeros((size, size), dtype=np.float32)
    kernel[size // 2, :] = 1
    kernel /= size
    return kernel
psf = motion_blur_kernel(15)
blurred = cv2.filter2D(img, -1, psf)
def inverse_filter(blurred, psf, threshold=0.01):
    G = np.fft.fft2(blurred)
    H = np.fft.fft2(psf, s=blurred.shape)
    H[np.abs(H) < threshold] = threshold
    F = G / H
    restored = np.fft.ifft2(F)
    restored = np.real(restored)
    restored = cv2.normalize(restored, None, 0, 1, cv2.NORM_MINMAX)
    return restored
restored = inverse_filter(blurred, psf)
plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Image")
plt.axis("off")
plt.subplot(1, 3, 2)
plt.imshow(blurred, cmap="gray")
plt.title("Blurred Image")
plt.axis("off")
plt.subplot(1, 3, 3)
```

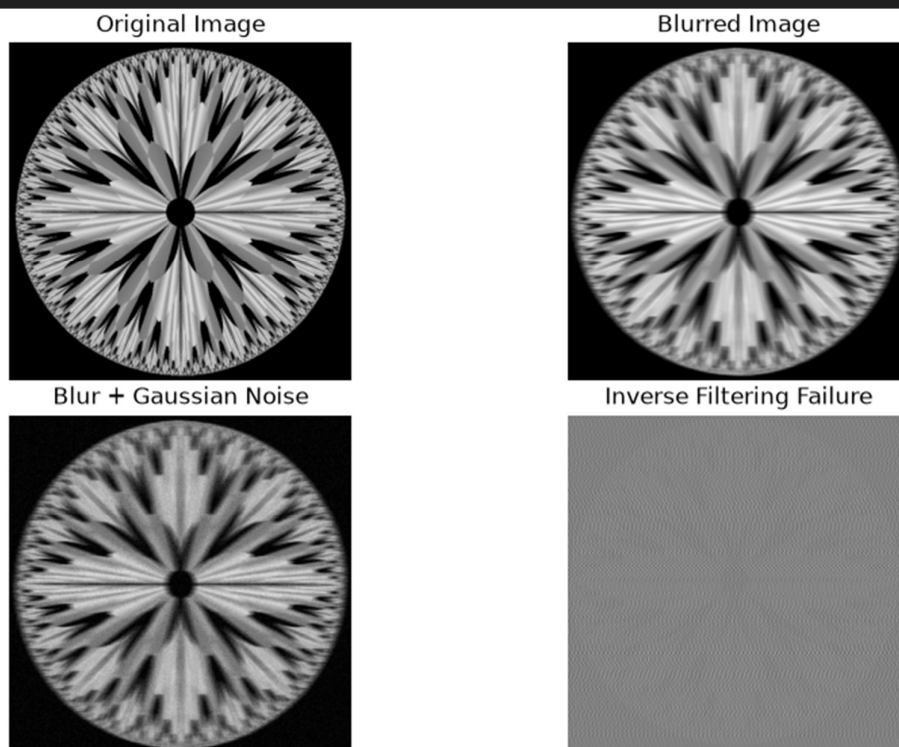
```
plt.imshow(restored, cmap="gray")
plt.title("Inverse Filtered Image")
plt.axis("off")
plt.tight_layout()
plt.show()
```



5. Worst Case of Inverse Filtering

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
img = cv2.imread("img_1.jpeg", cv2.IMREAD_GRAYSCALE)
if img is None: raise FileNotFoundError("Image not found")
img = img.astype(np.float32) / 255.0
def motion_blur_kernel(size=15):
    kernel = np.zeros((size, size), dtype=np.float32)
    kernel[size // 2, :] = 1
    kernel /= size
    return kernel
psf = motion_blur_kernel(15)
blurred = cv2.filter2D(img, -1, psf)
noise = np.random.normal(0, 0.05, blurred.shape)
degraded = blurred + noise
degraded = np.clip(degraded, 0, 1)
G = np.fft.fft2(degraded)
H = np.fft.fft2(psf, s=degraded.shape)
H[np.abs(H) < 1e-10] = 1e-10
F = G / H
restored = np.fft.ifft2(F)
restored = np.real(restored)
restored = cv2.normalize(restored, None, 0, 1, cv2.NORM_MINMAX)
plt.figure(figsize=(12, 8))
plt.subplot(2, 2, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Image")
```

```
plt.axis("off")
plt.subplot(2, 2, 2)
plt.imshow(blurred, cmap="gray")
plt.title("Blurred Image")
plt.axis("off")
plt.subplot(2, 2, 3)
plt.imshow(degraded, cmap="gray")
plt.title("Blur + Gaussian Noise")
plt.axis("off")
plt.subplot(2, 2, 4)
plt.imshow(restored, cmap="gray")
plt.title("Inverse Filtering Failure")
plt.axis("off")
plt.tight_layout()
plt.show()
```



6. Weiner Filter

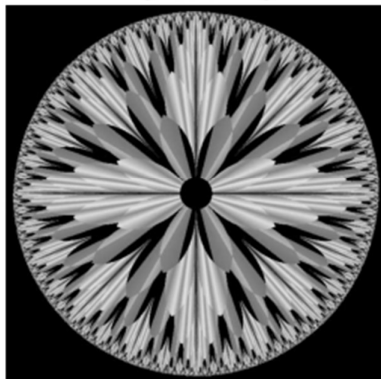
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
img = cv2.imread("img_1.jpeg", cv2.IMREAD_GRAYSCALE)
if img is None: raise FileNotFoundError("Image not found")
img = img.astype(np.float32) / 255.0
def motion_blur_kernel(size=15):
    kernel = np.zeros((size, size), dtype=np.float32)
    kernel[size // 2, :] = 1
    kernel /= size
    return kernel
psf = motion_blur_kernel(15)
blurred = cv2.filter2D(img, -1, psf)
noise = np.random.normal(0, 0.03, blurred.shape)
degraded = blurred + noise
degraded = np.clip(degraded, 0, 1)
def wiener_filter(degraded, psf, K=0.01):
    G = np.fft.fft2(degraded)
```

```

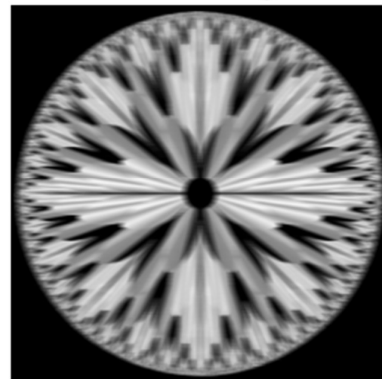
H = np.fft.fft2(psf, s=degraded.shape)
H_conj = np.conj(H)
W = H_conj / (np.abs(H) ** 2 + K)
F_hat = W * G
restored = np.fft.ifft2(F_hat)
restored = np.real(restored)
restored = cv2.normalize(restored, None, 0, 1, cv2.NORM_MINMAX)
return restored
restored = wiener_filter(degraded, psf, K=0.01)
plt.figure(figsize=(12, 8))
plt.subplot(2, 2, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Image")
plt.axis("off")
plt.subplot(2, 2, 2)
plt.imshow(blurred, cmap="gray")
plt.title("Blurred Image")
plt.axis("off")
plt.subplot(2, 2, 3)
plt.imshow(degraded, cmap="gray")
plt.title("Blur + Gaussian Noise")
plt.axis("off")
plt.subplot(2, 2, 4)
plt.imshow(restored, cmap="gray")
plt.title("Wiener Filtered Image")
plt.axis("off")
plt.tight_layout()
plt.show()

```

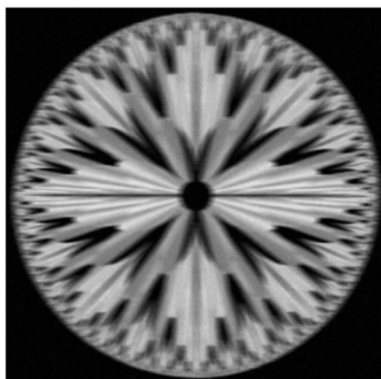
Original Image



Blurred Image



Blur + Gaussian Noise



Wiener Filtered Image

